

Highly Available 3-Tier Web Infrastructure on AWS

Custom VPC | Auto Scaling | Application Load Balancer | Route 53 | ACM (HTTPS)

Region: Asia Pacific (Mumbai) — ap-south-1
Domain: saurabhchavan.shop
Prepared by: Saurabh Chavan

Component	Details
VPC	prod-vpc — 10.0.0.0/16
Subnets	3 Public + 3 Private across 3 Availability Zones
Internet Access	Internet Gateway (public) + 3 NAT Gateways (private, one per AZ)
Compute	EC2 Auto Scaling Group (t3.micro), Min 1 / Desired 3 / Max 7
Load Balancing	Application Load Balancer (Internet-facing) — HTTP & HTTPS
Security	ALB-SG and EC2-SG security groups (VPC firewall)
DNS & SSL	Route 53 hosted zone + AWS Certificate Manager (ACM) certificate

Project Overview

This project provisions a production-style, highly available web application infrastructure on AWS, built entirely from scratch using a custom Virtual Private Cloud (VPC). The design follows the standard 3-tier public/private network pattern: web servers run in private subnets behind an internet-facing Application Load Balancer, outbound internet access for the private tier is provided through redundant NAT Gateways, and the whole environment is spread across three Availability Zones for fault tolerance. An Auto Scaling Group keeps the desired number of EC2 instances running automatically, and the final application is published on a custom domain secured with HTTPS.

Architecture at a Glance

Internet → Route 53 (DNS) → Application Load Balancer (public subnets, ports 80/443) → Target Group → EC2 instances in an Auto Scaling Group (private subnets, 3 AZs) → outbound traffic routed via per-AZ NAT Gateways → Internet Gateway.

Goals Achieved

- Isolated, custom-built network (no default VPC) with clear public/private separation.
- Multi-AZ deployment (ap-south-1a, 1b, 1c) for high availability.
- Zero single point of failure for outbound internet access (1 NAT Gateway per AZ).
- Automatic scaling of compute capacity based on a configured Auto Scaling Group.
- Centralized traffic distribution and health checking via an Application Load Balancer.
- Custom domain with DNS routing (Route 53) and TLS/HTTPS encryption (AWS Certificate Manager).

Phase 1: Networking Foundation (Custom VPC)

A dedicated VPC was created to fully control the network layout, instead of using the AWS default VPC. The VPC spans three Availability Zones, each with one public and one private subnet, giving a resilient 6-subnet layout that separates internet-facing resources from backend compute.

Step 1: Create the VPC

Created a new VPC named **prod-vpc** with the IPv4 CIDR block **10.0.0.0/16**, using the "VPC only" option so subnets, route tables, and gateways could be added manually for full control.

The screenshot displays the AWS Management Console interface for creating a new VPC. The breadcrumb navigation shows 'VPC > Your VPCs > Create VPC'. The main heading is 'Create VPC' with an 'Info' link. Below this, a descriptive sentence states: 'A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.'

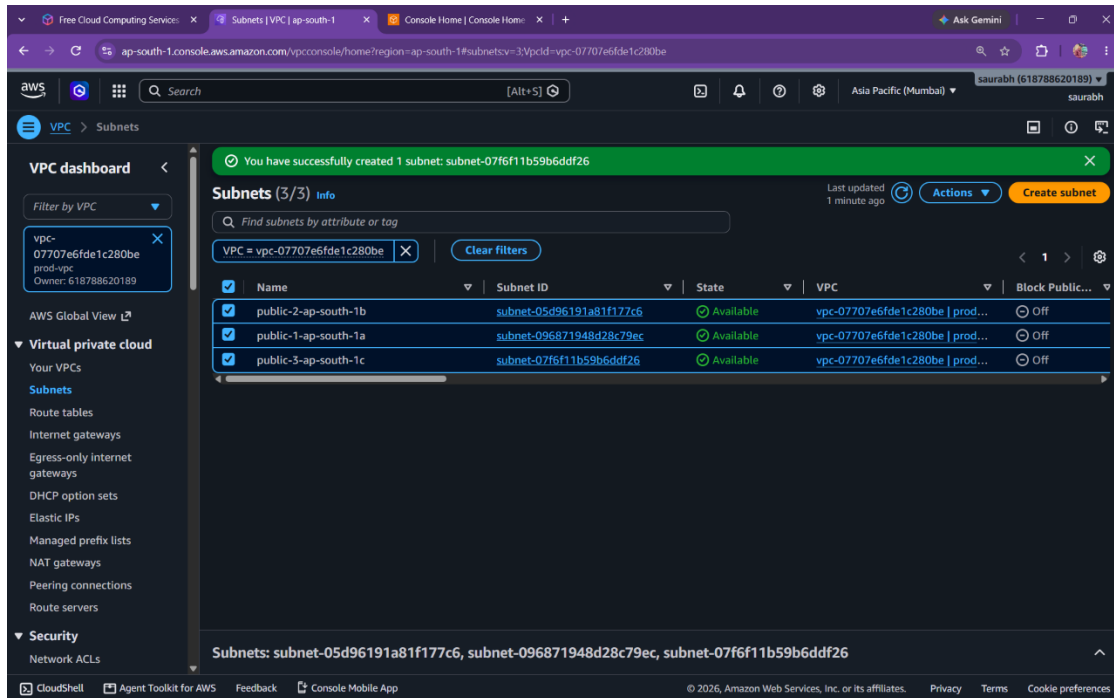
The 'VPC settings' section contains the following configuration options:

- Resources to create:** Two radio buttons are present. 'VPC only' is selected, while 'VPC and more' is unselected.
- Name tag - optional:** A text input field contains the value 'prod-vpc'. A note below the field states: 'Creates a tag with a key of "Name" and a value that you specify.'
- IPv4 CIDR block:** Two radio buttons are present. 'IPv4 CIDR manual input' is selected, while 'IPAM-allocated IPv4 CIDR block' is unselected. Below this, a text input field contains the value '10.0.0.0/16'. A note below the field states: 'CIDR block size must be between /16 and /28.'
- IPv6 CIDR block:** Four radio buttons are present. 'No IPv6 CIDR block' is selected, while 'IPAM-allocated IPv6 CIDR block', 'Amazon-provided IPv6 CIDR block', and 'IPv6 CIDR owned by me' are unselected.
- Tenancy:** A dropdown menu is set to 'Default'.

The footer of the console shows links for 'CloudShell', 'Agent Toolkit for AWS', 'Feedback', and 'Console Mobile App', along with copyright information and links for 'Privacy', 'Terms', and 'Cookie preferences'.

Step 2: Create Public Subnets

Three public subnets were created, one per Availability Zone: **public-1-ap-south-1a**, **public-2-ap-south-1b**, and **public-3-ap-south-1c**. These subnets will host the Load Balancer and NAT Gateways.



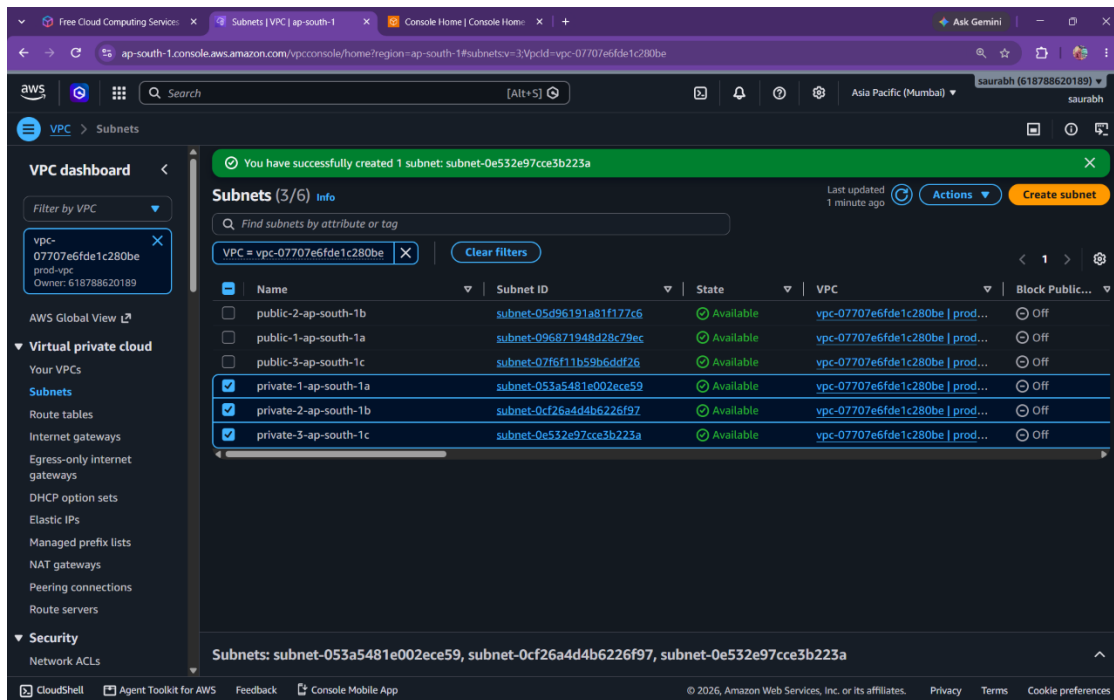
The screenshot displays the AWS Management Console's VPC dashboard. A green notification banner at the top states: "You have successfully created 1 subnet: subnet-07f6f11b59b6ddf26". The main section, titled "Subnets (3/3)", shows a list of three subnets, all in an "Available" state, associated with VPC "vpc-07707e6fde1c280be".

Name	Subnet ID	State	VPC	Block Public...
public-2-ap-south-1b	subnet-05d96191a81f177c6	Available	vpc-07707e6fde1c280be prod...	Off
public-1-ap-south-1a	subnet-096871948d28c79ec	Available	vpc-07707e6fde1c280be prod...	Off
public-3-ap-south-1c	subnet-07f6f11b59b6ddf26	Available	vpc-07707e6fde1c280be prod...	Off

The left sidebar shows the navigation menu with categories like "Virtual private cloud" and "Security". The bottom of the console shows the footer with copyright information and links to CloudShell, Agent Toolkit, and other resources.

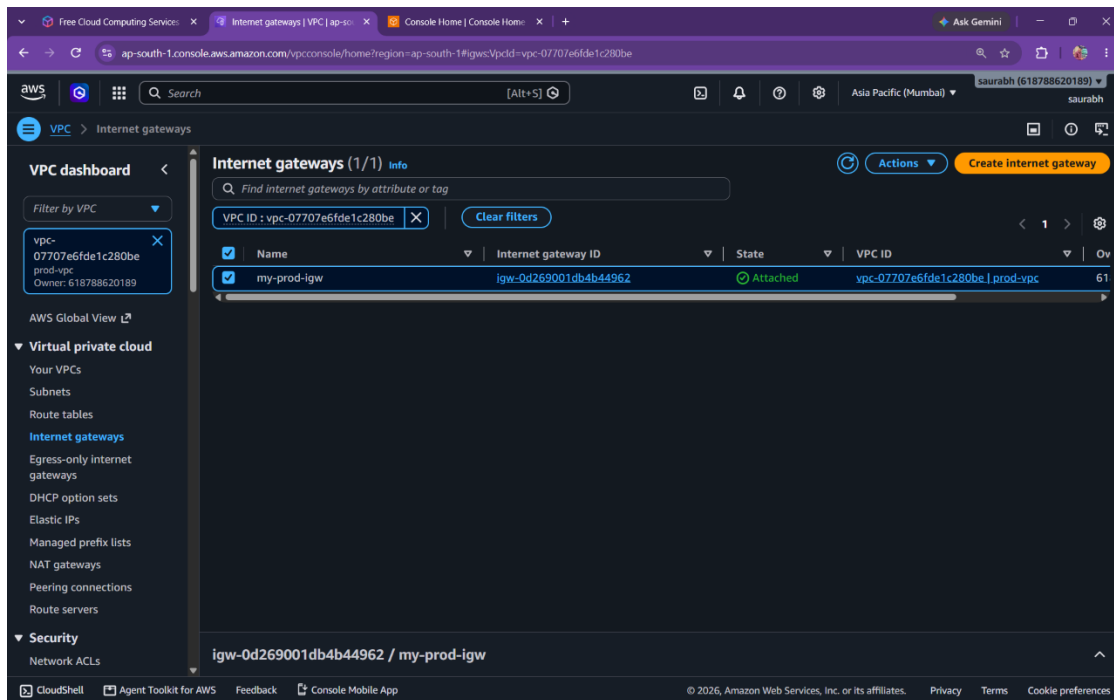
Step 3: Create Private Subnets

Three matching private subnets were created — **private-1-ap-south-1a**, **private-2-ap-south-1b**, and **private-3-ap-south-1c** — bringing the total to 6 subnets. The private subnets host the EC2 application instances, keeping them unreachable directly from the internet.



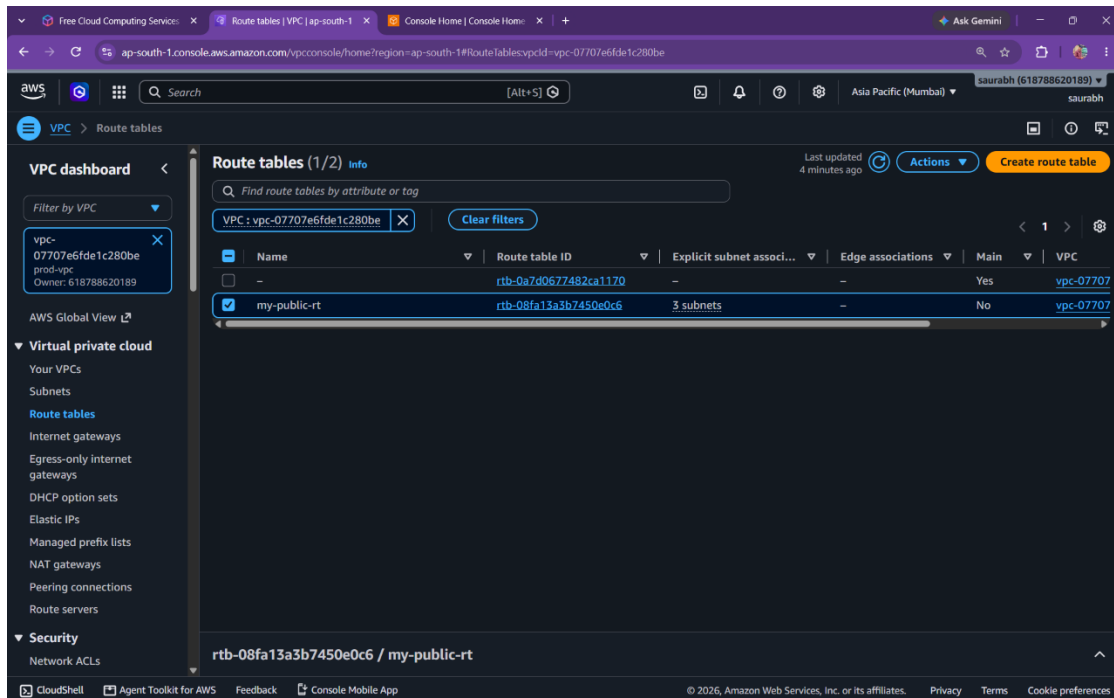
Step 4: Create and Attach an Internet Gateway

An Internet Gateway named **my-prod-igw** was created and attached to prod-vpc, giving the VPC a path to and from the public internet.



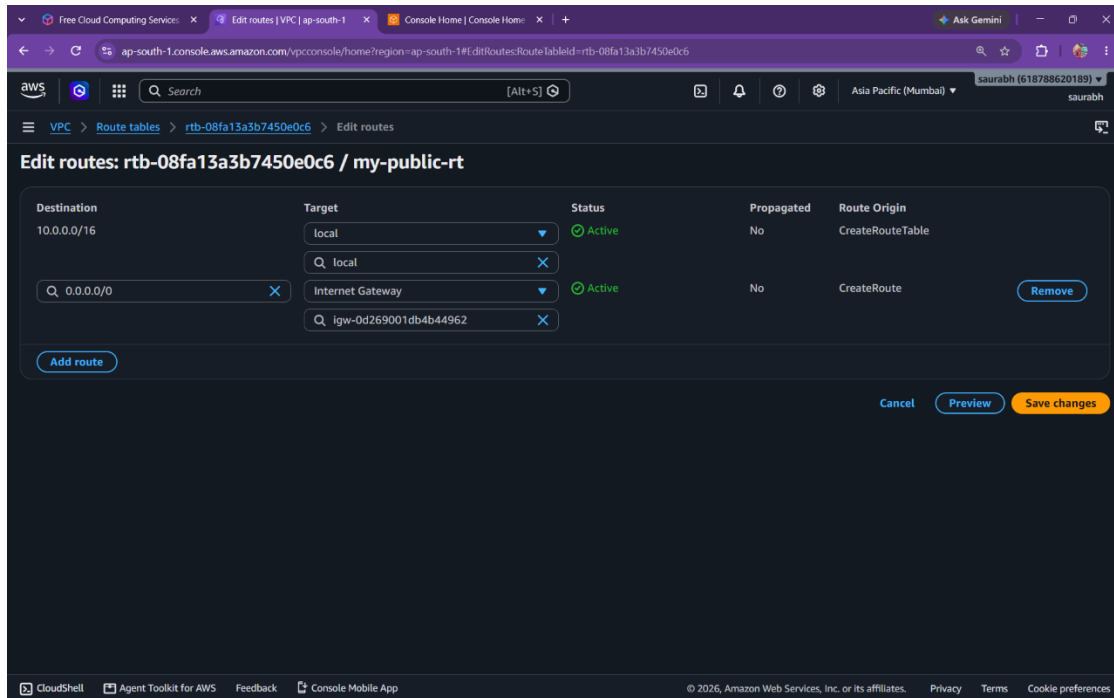
Step 5: Create the Public Route Table

A route table named **my-public-rt** was created for the public tier.



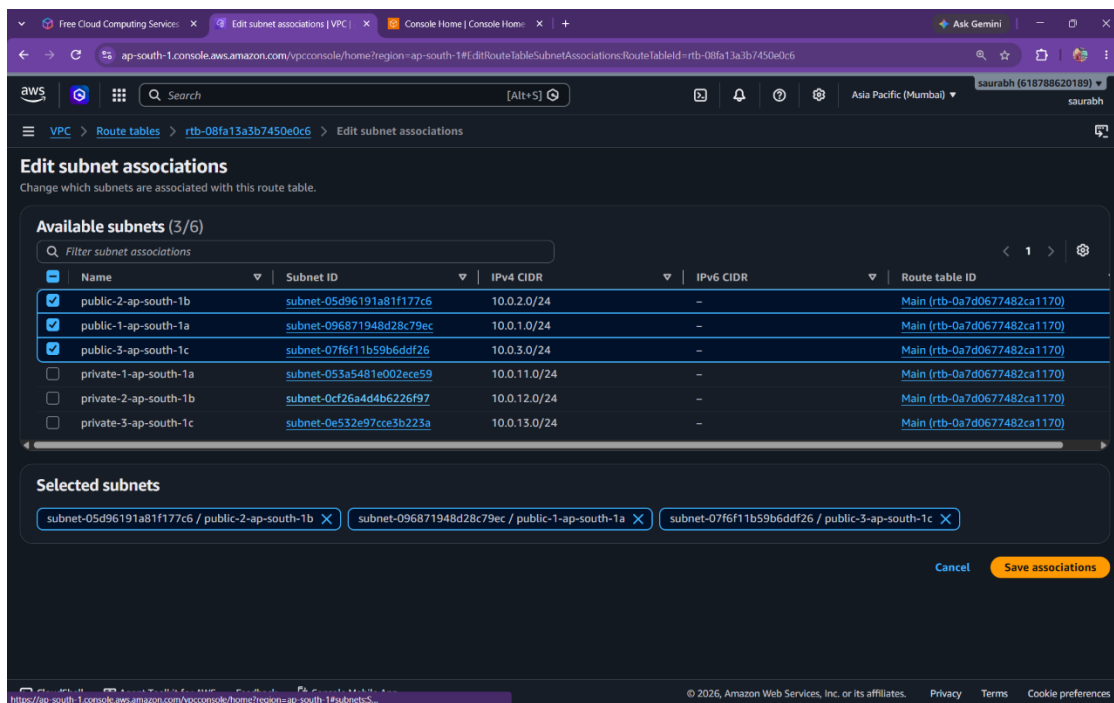
Step 6: Route Public Traffic to the Internet Gateway

In my-public-rt, a route was added sending all traffic (**0.0.0.0/0**) to the Internet Gateway (igw-0d269001db4b44962), while local VPC traffic (10.0.0.0/16) stays on the local route.



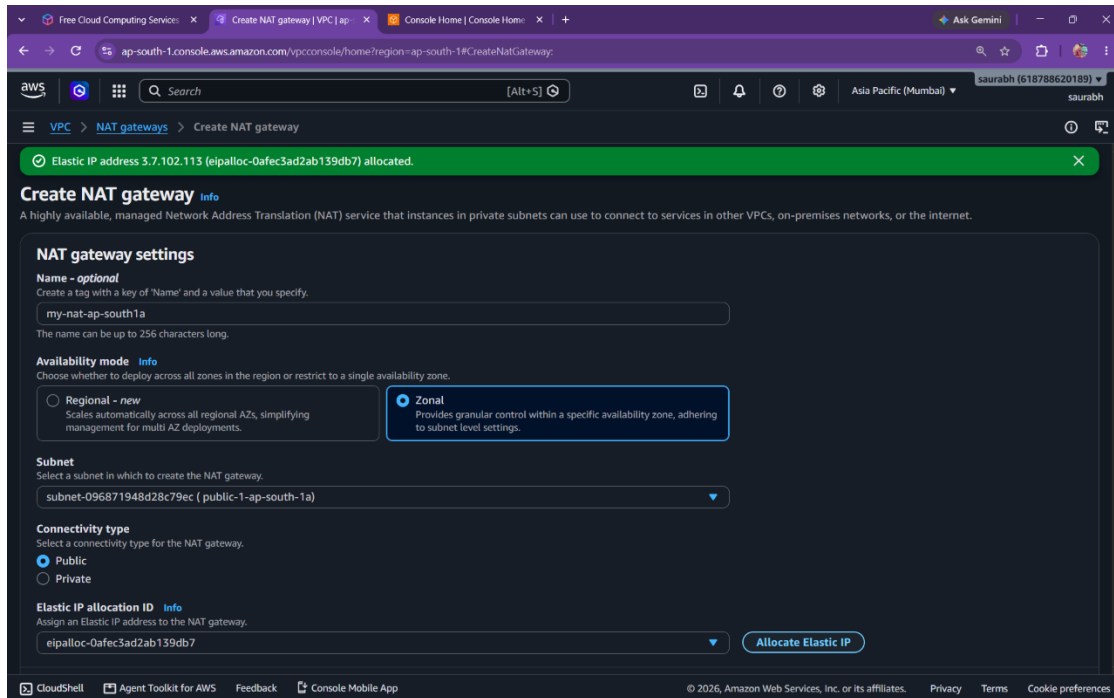
Step 7: Associate Public Subnets with the Public Route Table

All three public subnets (public-1, public-2, public-3) were explicitly associated with my-public-rt so resources in those subnets use the Internet Gateway for outbound/inbound traffic.



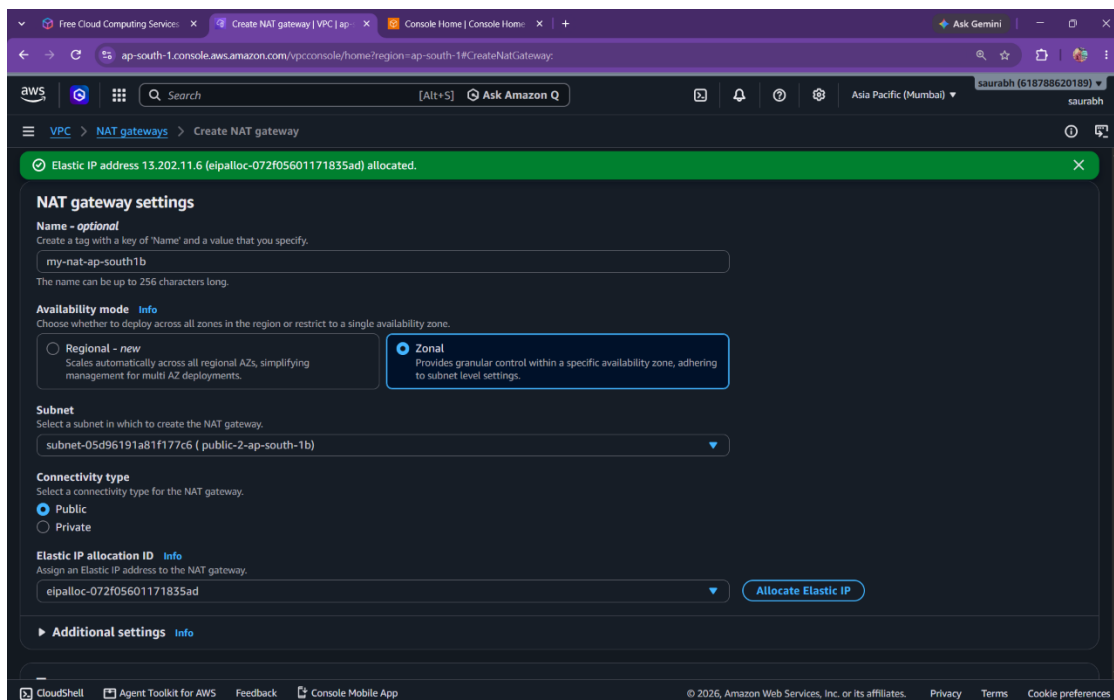
Step 8: Allocate Elastic IPs and Create NAT Gateway (AZ-a)

A NAT Gateway **my-nat-ap-south1a** was created inside public-1-ap-south-1a with a newly allocated Elastic IP (3.7.102.113), giving instances in the AZ-a private subnet outbound internet access.



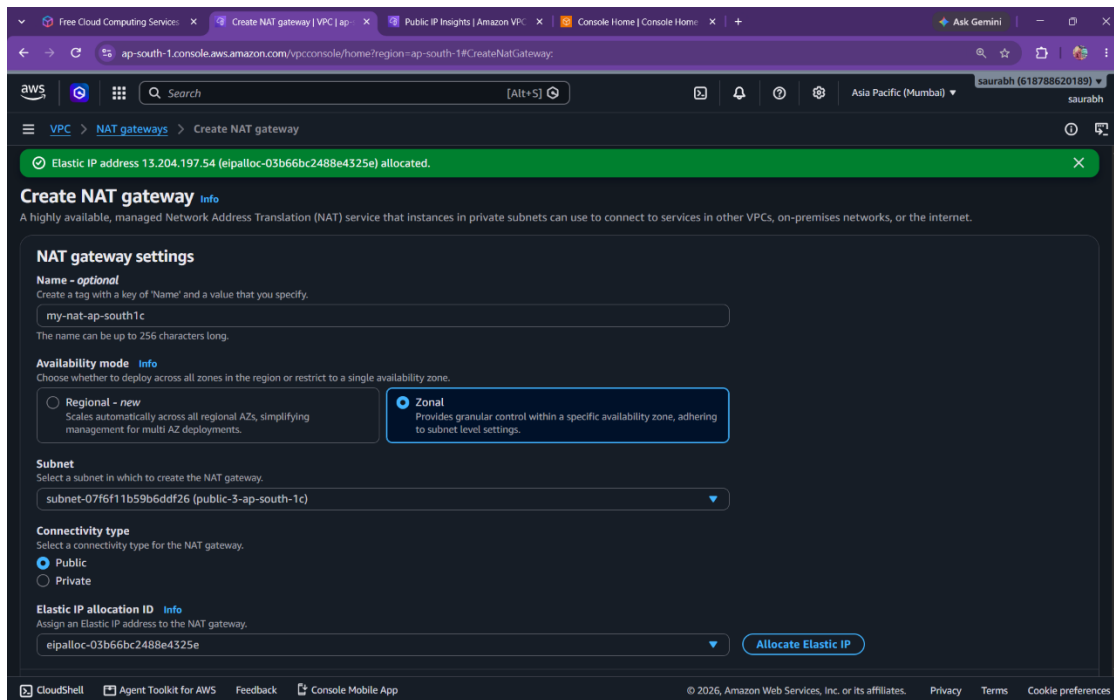
Step 9: Create NAT Gateway (AZ-b)

A second NAT Gateway **my-nat-ap-south1b** was created in public-2-ap-south-1b with its own Elastic IP (13.202.11.6), so each Availability Zone has an independent, fault-isolated NAT path.



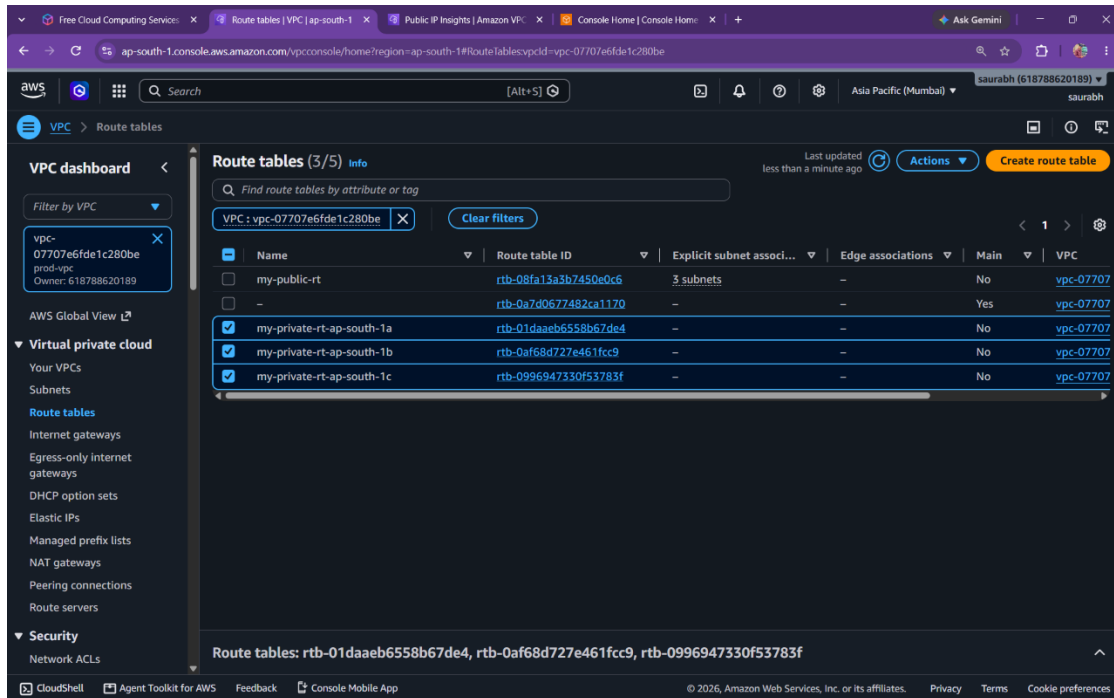
Step 10: Create NAT Gateway (AZ-c)

A third NAT Gateway **my-nat-ap-south1c** was created in public-3-ap-south-1c with Elastic IP 13.204.197.54, completing one NAT Gateway per Availability Zone — this design avoids a single point of failure and reduces cross-AZ data transfer.



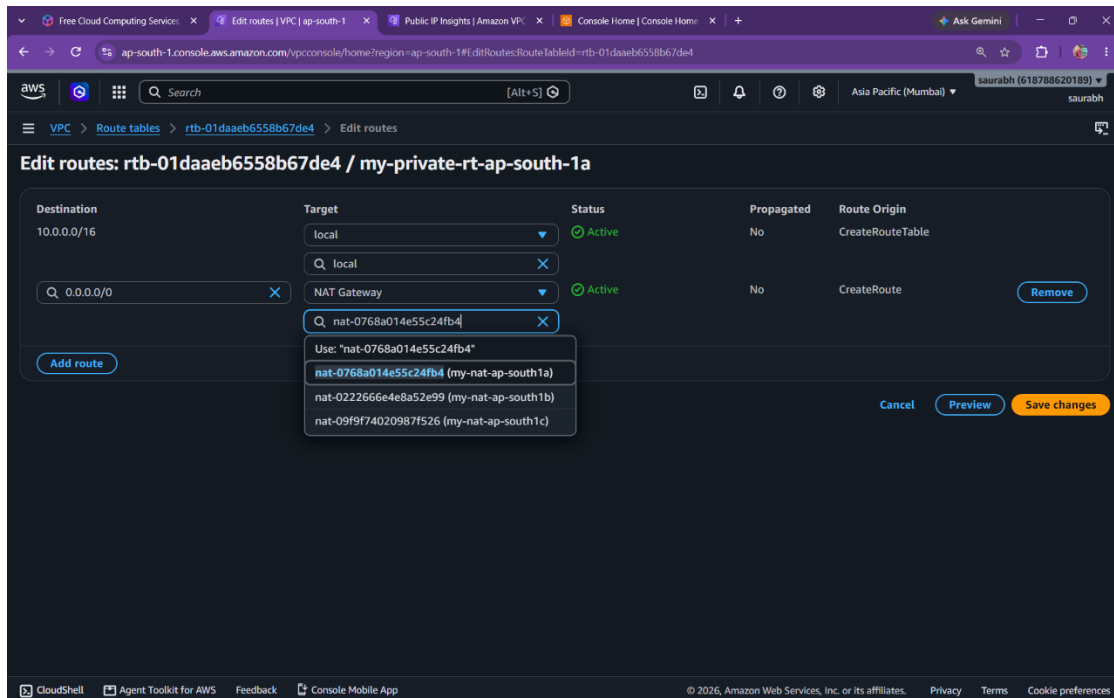
Step 11: Create Three Private Route Tables

Three separate private route tables were created — **my-private-rt-ap-south-1a/1b/1c** — one per AZ, so each private subnet's outbound traffic can be pinned to the NAT Gateway in its own zone.



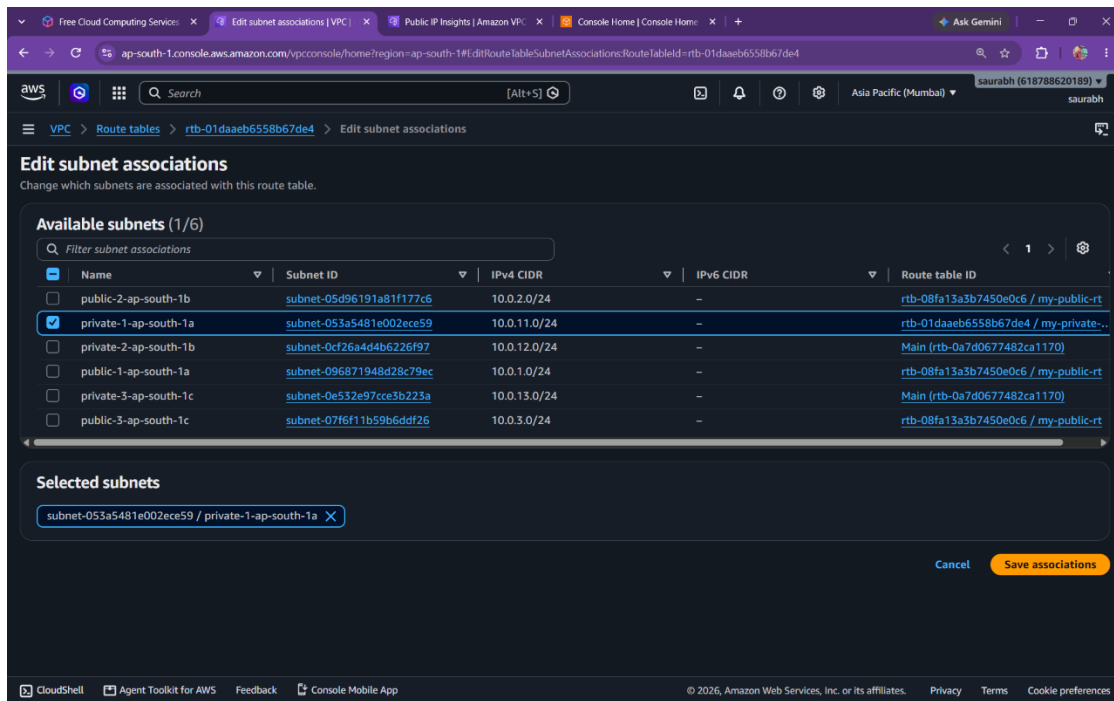
Step 12: Route Private Subnet (AZ-a) Traffic via its NAT Gateway

In my-private-rt-ap-south-1a, a 0.0.0.0/0 route was pointed at nat-0768a014e55c24fb4 (my-nat-ap-south1a).



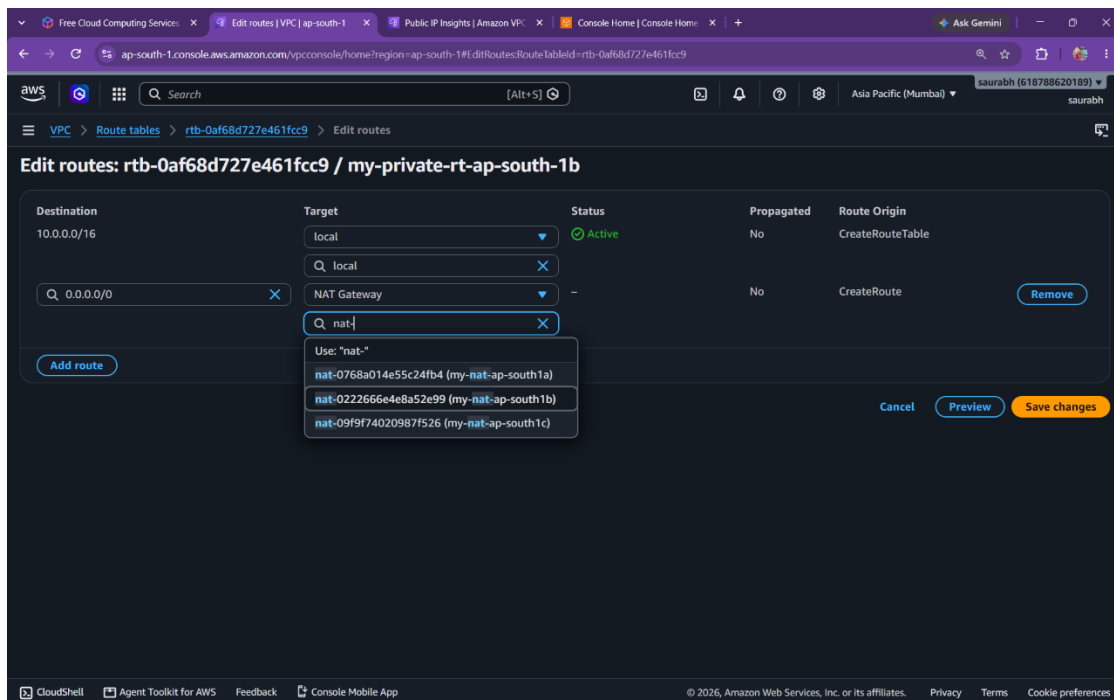
Step 13: Associate Private Subnet (AZ-a)

private-1-ap-south-1a was associated with my-private-rt-ap-south-1a.



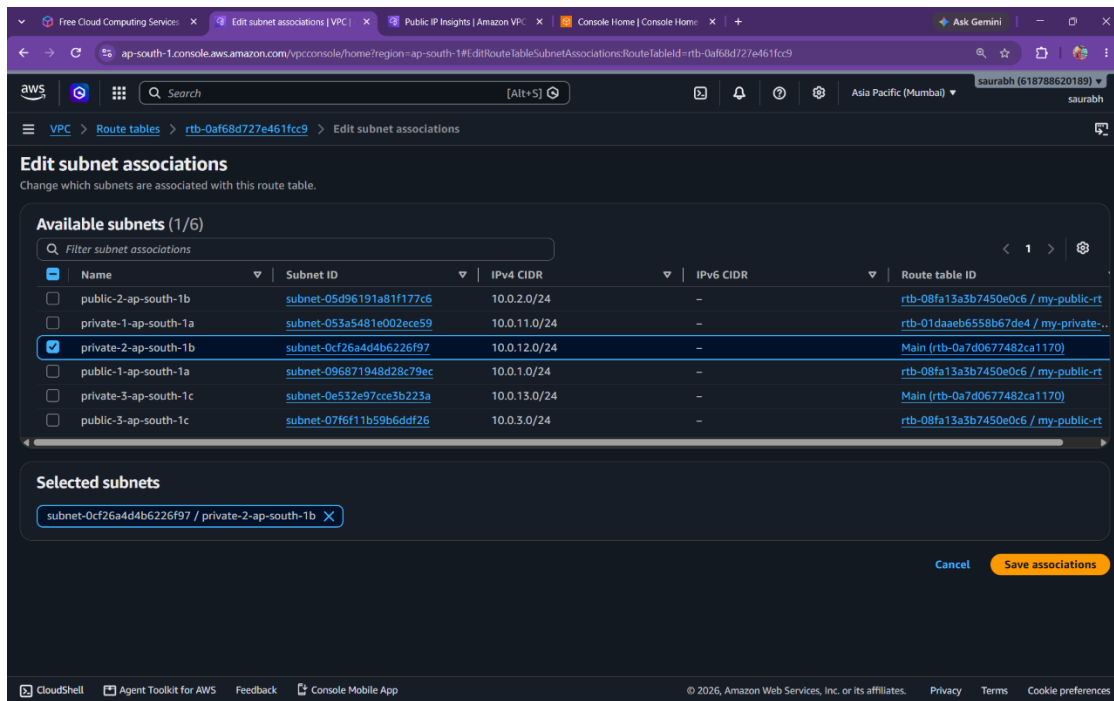
Step 14: Route Private Subnet (AZ-b) Traffic via its NAT Gateway

In my-private-rt-ap-south-1b, a 0.0.0.0/0 route was pointed at nat-0222666e4e8a52e99 (my-nat-ap-south1b).



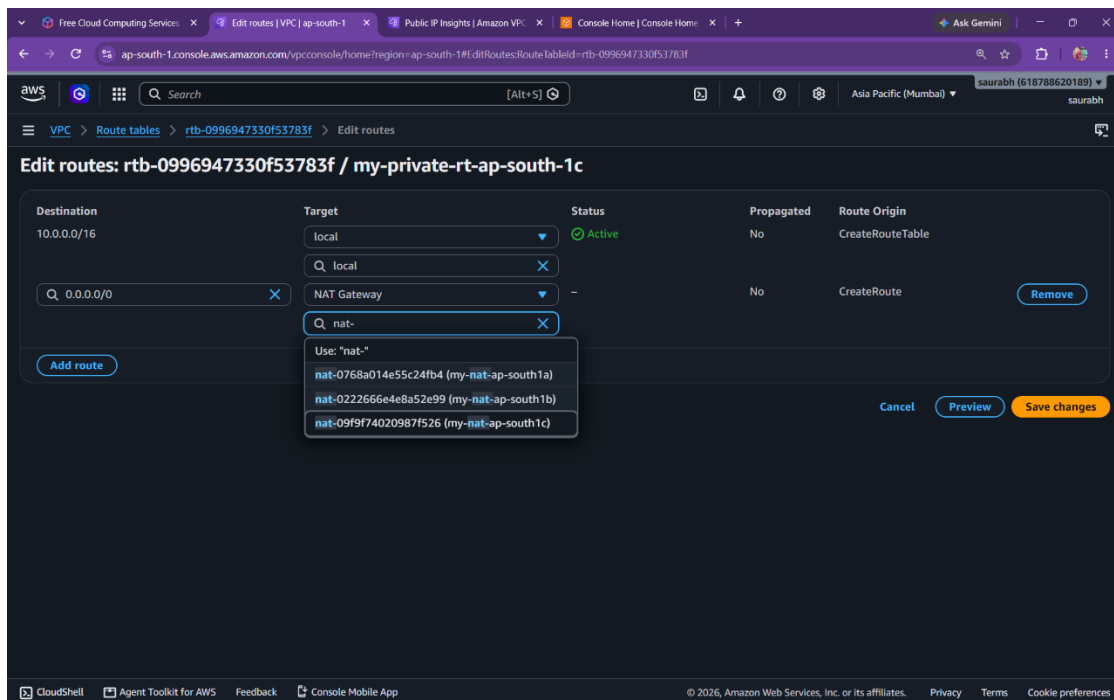
Step 15: Associate Private Subnet (AZ-b)

private-2-ap-south-1b was associated with my-private-rt-ap-south-1b.



Step 16: Route Private Subnet (AZ-c) Traffic via its NAT Gateway

In my-private-rt-ap-south-1c, a 0.0.0.0/0 route was pointed at nat-09f9f74020987f526 (my-nat-ap-south1c).



Step 17: Associate Private Subnet (AZ-c)

private-3-ap-south-1c was associated with my-private-rt-ap-south-1c, completing symmetric per-AZ routing for all three private subnets.

Edit subnet associations
Change which subnets are associated with this route table.

Available subnets (1/6)

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	public-2-ap-south-1b	subnet-05d96191a81f177c6	10.0.2.0/24	-	rtb-08fa13a3b7450e0c6 / my-public-rt
☐	private-1-ap-south-1a	subnet-053a5481e002ece59	10.0.11.0/24	-	rtb-01daaeb6558b67de4 / my-private-...
☐	private-2-ap-south-1b	subnet-0cf26a4d4b6226f97	10.0.12.0/24	-	rtb-0af68d727e461fcc9 / my-private-r...
☐	public-1-ap-south-1a	subnet-096871948d28c79ec	10.0.1.0/24	-	rtb-08fa13a3b7450e0c6 / my-public-rt
☒	private-3-ap-south-1c	subnet-0e532e97cce3b223a	10.0.13.0/24	-	Main (rtb-0a7d0677482ca1170)
☐	public-3-ap-south-1c	subnet-07f6f11b59b6ddf26	10.0.3.0/24	-	rtb-08fa13a3b7450e0c6 / my-public-rt

Selected subnets

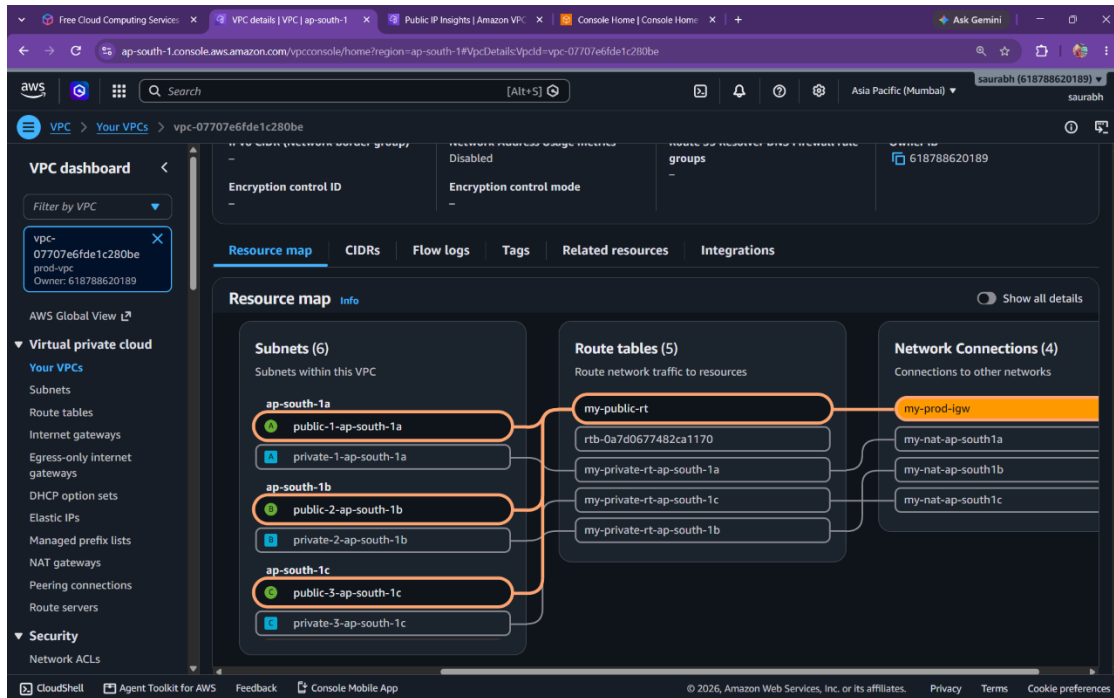
subnet-0e532e97cce3b223a / private-3-ap-south-1c

[Cancel](#) [Save associations](#)

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

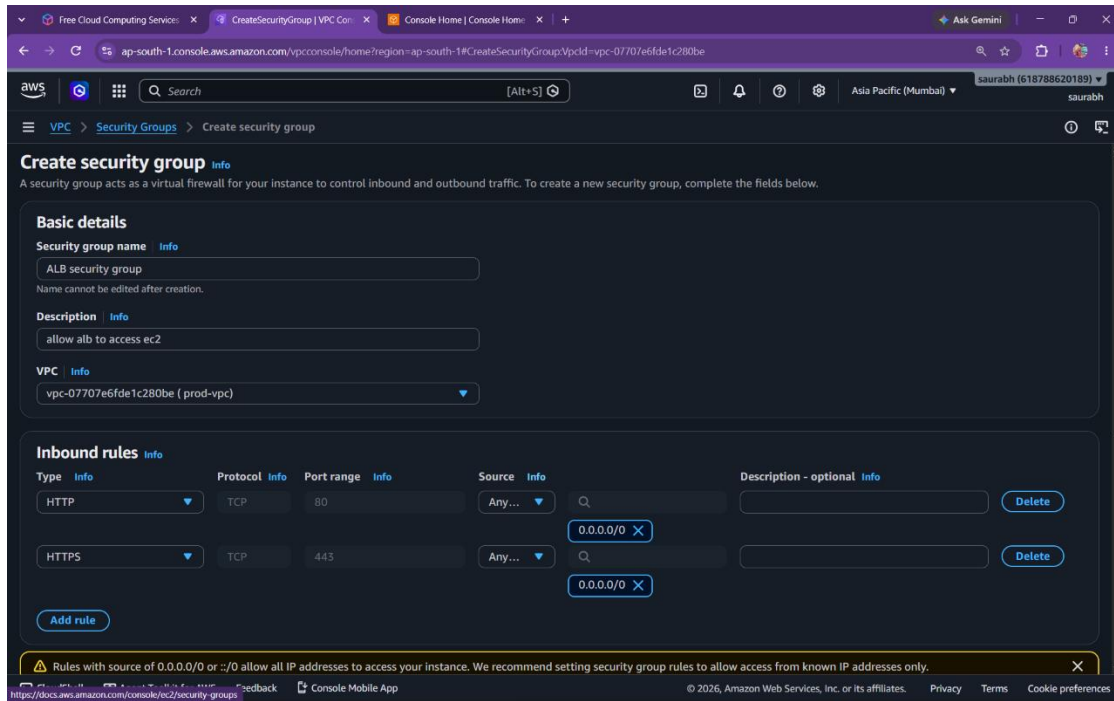
Step 18: Verify the VPC Resource Map

The VPC resource map confirms the final topology: 6 subnets across 3 AZs, 5 route tables (1 public + 3 private + default), and 4 network connections (1 Internet Gateway + 3 NAT Gateways) — matching the intended design end-to-end.



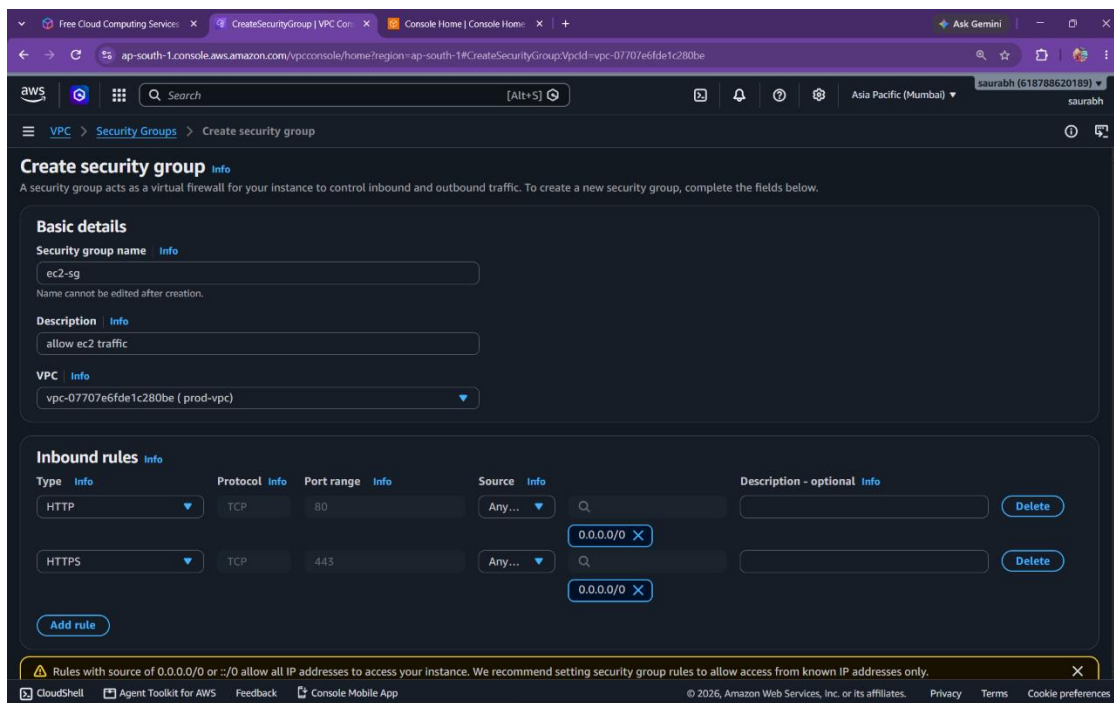
Step 19: Create the ALB Security Group

A security group **ALB-security-group** was created in prod-vpc allowing inbound HTTP (80) and HTTPS (443) from anywhere (0.0.0.0/0), acting as the public-facing firewall for the Load Balancer.



Step 20: Create the EC2 Security Group

A second security group **ec2-sg** was created to allow HTTP/HTTPS traffic to the web instances, completing the network-layer security setup before moving to compute provisioning.



Phase 2: Compute, Auto Scaling & Load Balancing

With the network in place, this phase provisions the application tier: a reusable EC2 launch template, a Target Group, an Application Load Balancer, and an Auto Scaling Group that automatically launches instances across all three private subnets and registers them behind the load balancer.

Step 21: Create an EC2 Launch Template

A launch template named **web-template** (version v1) was created to standardize the AMI, instance type (t3.micro), security group, and storage configuration used by every Auto Scaling instance.

The screenshot displays the AWS Management Console interface for creating a new EC2 launch template. The browser tabs at the top include 'Free Cloud Computing Services', 'VPC | ap-south-1', and 'Create launch template | EC2 |'. The URL bar shows 'ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#CreateLaunchTemplate:'. The console header shows the AWS logo, a search bar, and the user's profile 'saurabh (618788620189)'.

The main content area is titled 'Create launch template' with a subtitle: 'Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.' The breadcrumb navigation is 'EC2 > Launch templates > Create launch template'.

The 'Launch template name and description' section contains a text input for 'Launch template name - required' with the value 'web-template' and a note: 'Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '\', '@:'.' Below it is a text input for 'Template version description' with the value 'v1' and a note: 'Max 255 chars'.

The 'Auto Scaling guidance' section has a link to 'Info' and text: 'Select this if you intend to use this template with EC2 Auto Scaling'. There is an unchecked checkbox labeled 'Provide guidance to help me set up a template that I can use with EC2 Auto Scaling'.

Below this are expandable sections: 'Template tags' and 'Source template'.

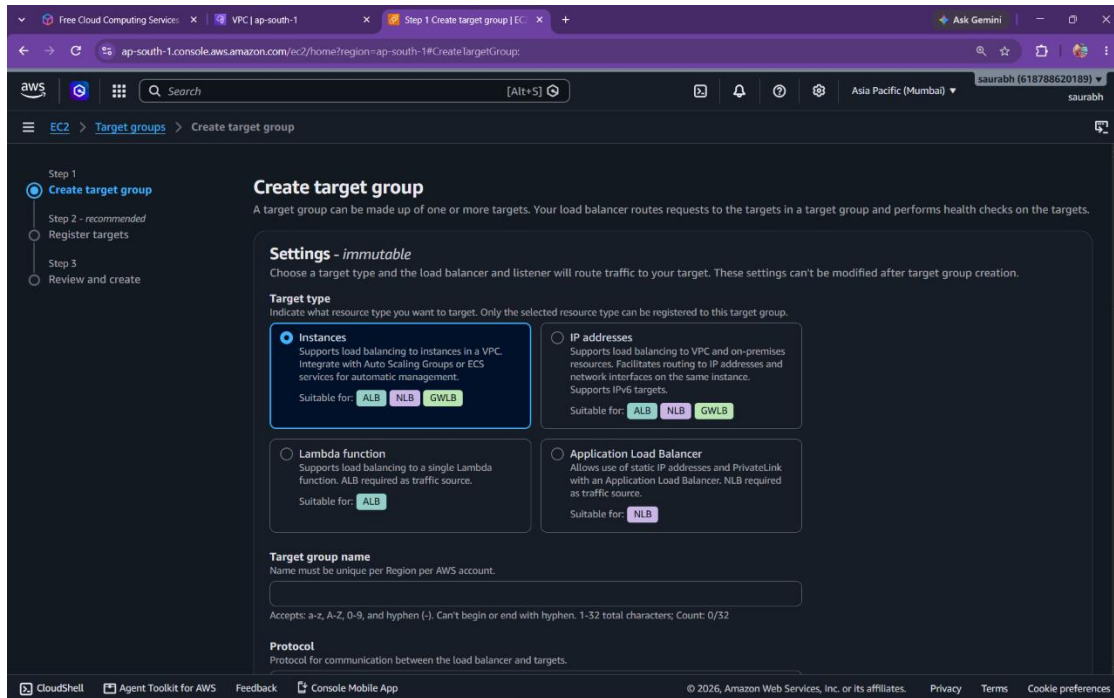
The 'Launch template contents' section has a subtitle: 'Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.' It features a section titled 'Application and OS Images (Amazon Machine Image)' with a link to 'Info' and a note: 'An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below,'.

On the right side, a 'Summary' panel lists the following items, each with a minus sign indicating it can be expanded: 'Software Image (AMI)', 'Virtual server type (instance type)', 'Firewall (security group)', and 'Storage (volumes)'. At the bottom of this panel are two buttons: 'Cancel' and 'Create launch template'.

The footer of the console includes links for 'CloudShell', 'Agent Toolkit for AWS', 'Feedback', and 'Console Mobile App', along with the copyright notice '© 2026, Amazon Web Services, Inc. or its affiliates.' and links for 'Privacy', 'Terms', and 'Cookie preferences'.

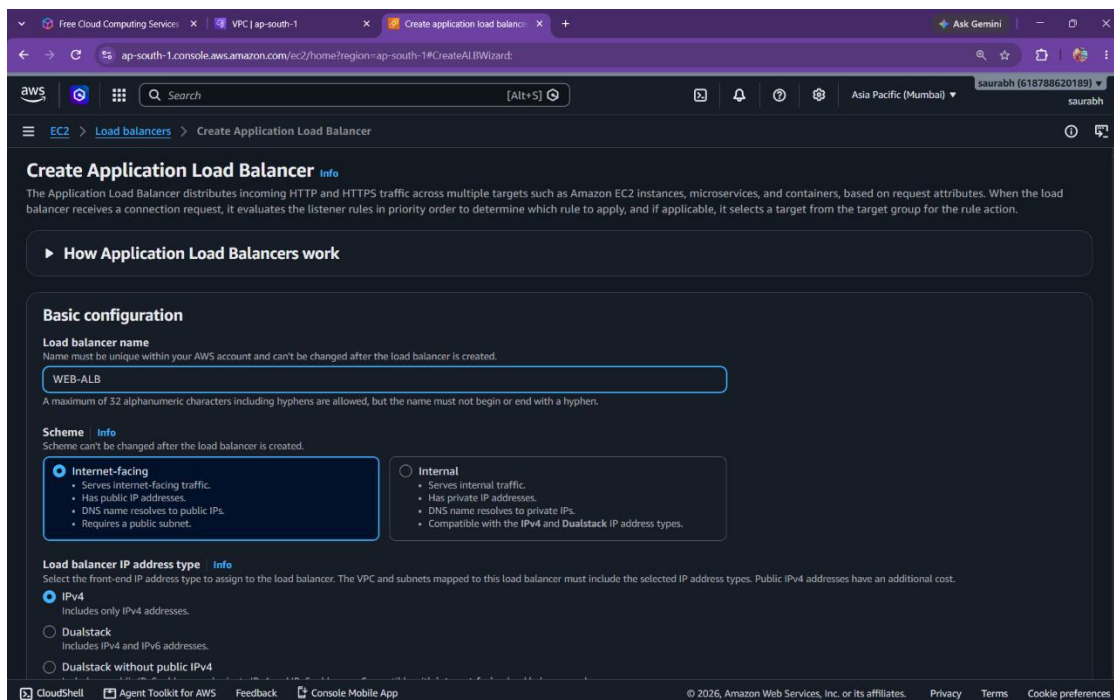
Step 22: Create a Target Group

A target group **web-tg** was created with target type "Instances", which the Load Balancer will use to route requests to healthy EC2 instances and run periodic health checks.



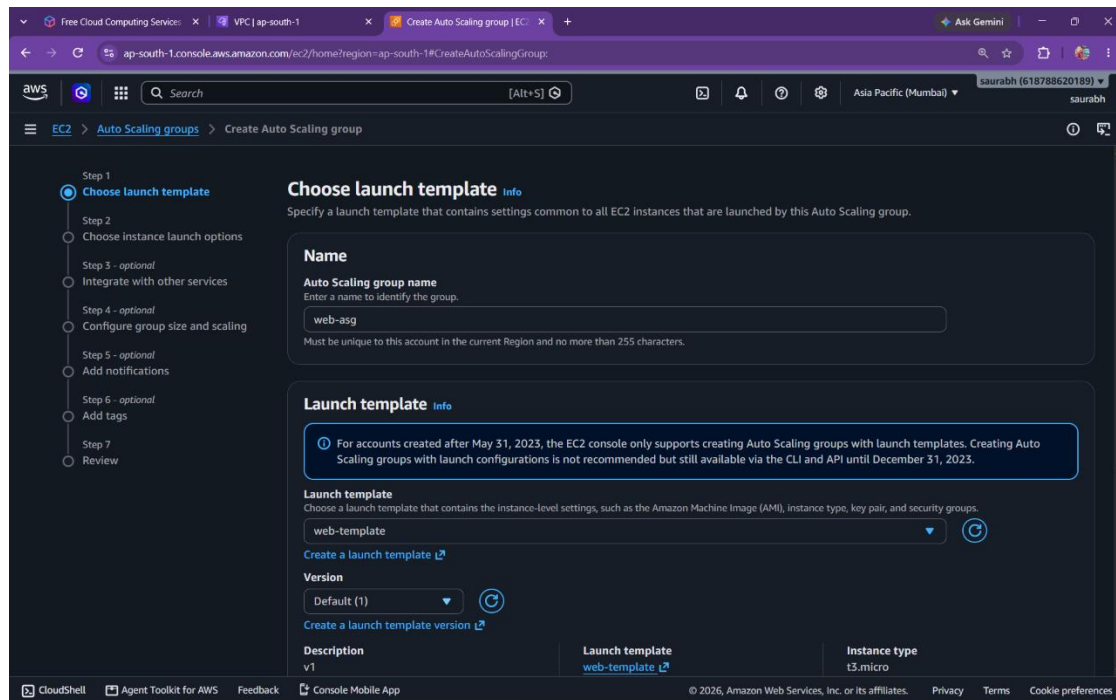
Step 23: Create the Application Load Balancer

An internet-facing Application Load Balancer named **WEB-ALB** was created with IPv4 addressing, designed to accept public traffic and forward it to the private EC2 instances.



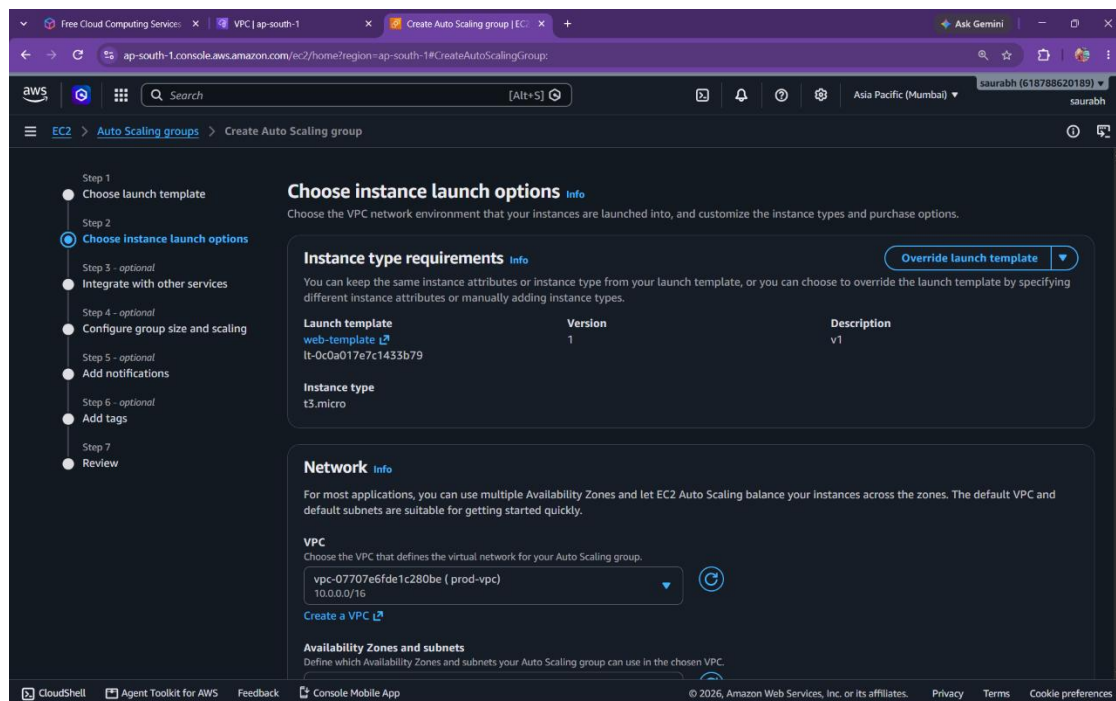
Step 24: Create the Auto Scaling Group — Choose Launch Template

An Auto Scaling Group named **web-asg** was created, based on the web-template launch template (t3.micro instances).



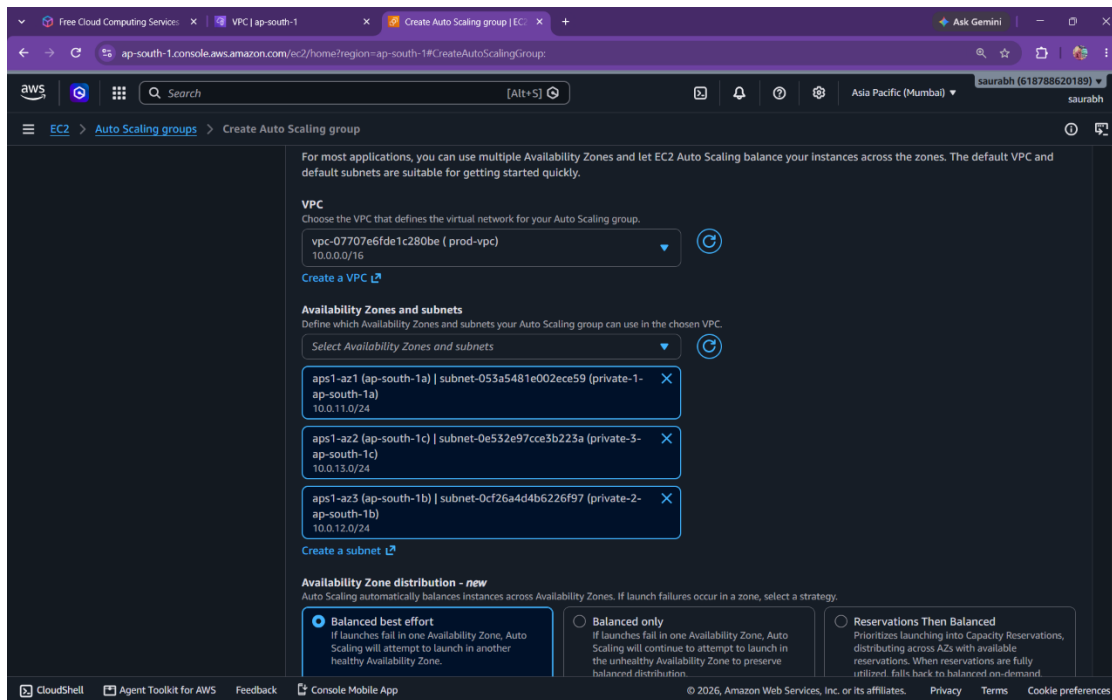
Step 25: Choose Instance Launch Options (VPC)

prod-vpc was selected as the network for the Auto Scaling group's instances.



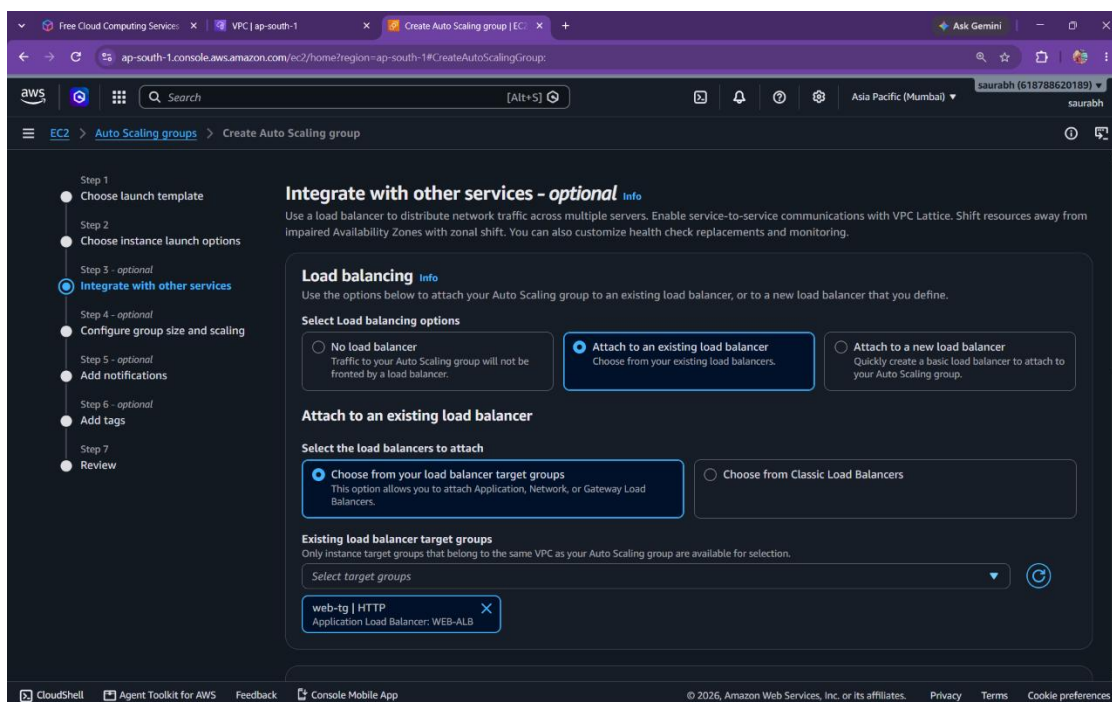
Step 26: Select Availability Zones and Subnets

All three private subnets — private-1-ap-south-1a, private-2-ap-south-1b, and private-3-ap-south-1c — were selected so the Auto Scaling Group distributes instances evenly across all AZs.



Step 27: Attach the Load Balancer Target Group

The ASG was integrated with the existing load balancer by attaching it to target group **web-tg**, so every instance it launches is automatically registered for traffic.



Step 28: Configure Group Size and Scaling

Desired capacity was set to **3** instances, with scaling limits of **Min 1 / Max 7**, allowing the group to shrink or grow automatically within that range as demand changes.

The screenshot shows the 'Configure group size and scaling - optional' step in the AWS Management Console. The left sidebar lists steps from 'Choose launch template' to 'Review', with 'Configure group size and scaling' selected. The main content area is divided into three sections: 'Group size', 'Desired capacity type', and 'Scaling'. In the 'Group size' section, 'Units (number of instances)' is set to 3. In the 'Desired capacity type' section, 'Units (number of instances)' is selected. In the 'Scaling' section, 'Min desired capacity' is set to 1 and 'Max desired capacity' is set to 7. The 'Automatic scaling - optional' section is also visible.

Configure group size and scaling - optional [Info](#)

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory (GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity

Specify your group size.

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity **Max desired capacity**

Equal or less than desired capacity Equal or greater than desired capacity

Automatic scaling - optional

Step 29: Auto Scaling Group Created

The web-arg Auto Scaling Group was created successfully and immediately began updating capacity toward the desired count of 3 instances.

The screenshot shows the 'Auto Scaling groups' page in the AWS Management Console. The top section shows a list of Auto Scaling groups with columns for Name, Status, Launch template/configuration, Instances, Instance health, and Desired capacity. The 'web-arg' group is highlighted, showing a status of 'Updating capacity' and a desired capacity of 3. Below the list, the 'Auto Scaling group: web-arg' details are shown, including the group's ARN, desired capacity, scaling limits, and status.

Auto Scaling groups (1/1) [Info](#)

Last updated less than a minute ago [Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

☒	Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity
☒	web-arg	Updating capacity	web-template Version 1	0	-	3

Auto Scaling group: web-arg

[Details](#) [Integrations](#) [Automatic scaling](#) [Instance management](#) [Instance refresh](#) [Activity](#) [Monitoring](#) [Tags](#)

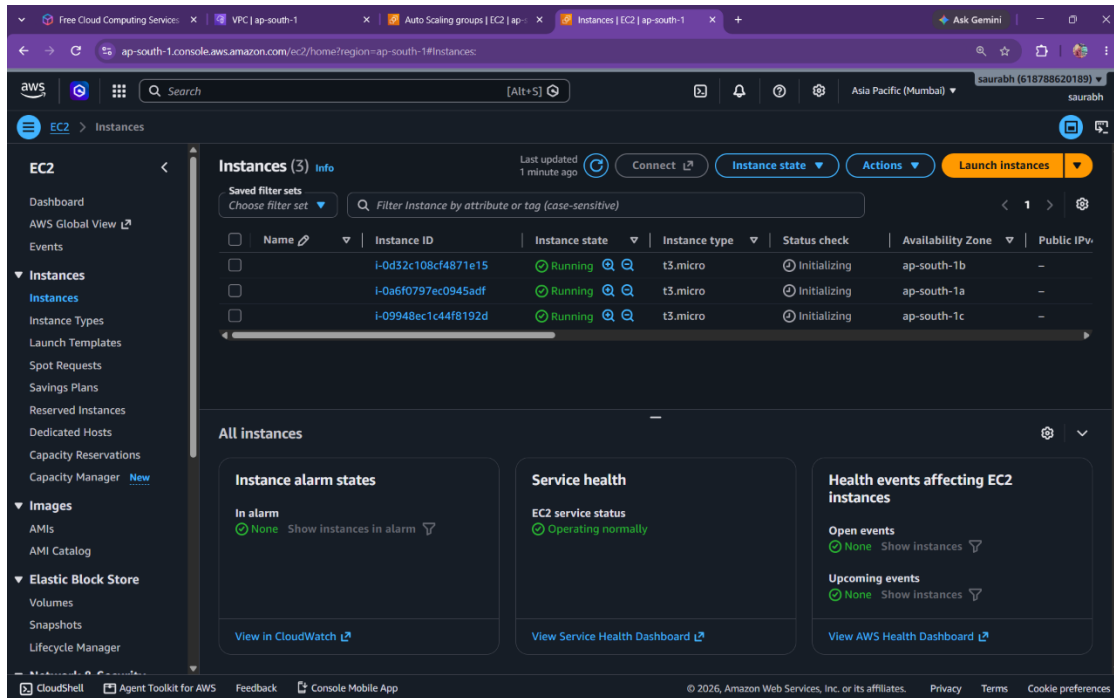
web-arg Capacity overview [Edit](#)

arn:aws:autoscaling:ap-south-1:618788620189:autoScalingGroup:39bb3e4e-15cf-457a-91b5-ceba53b777a5:autoScalingGroupName/web-arg

Desired capacity 3	Scaling limits 1 - 7	Desired capacity type Units (number of instances)	Status Updating capacity
------------------------------	--------------------------------	---	------------------------------------

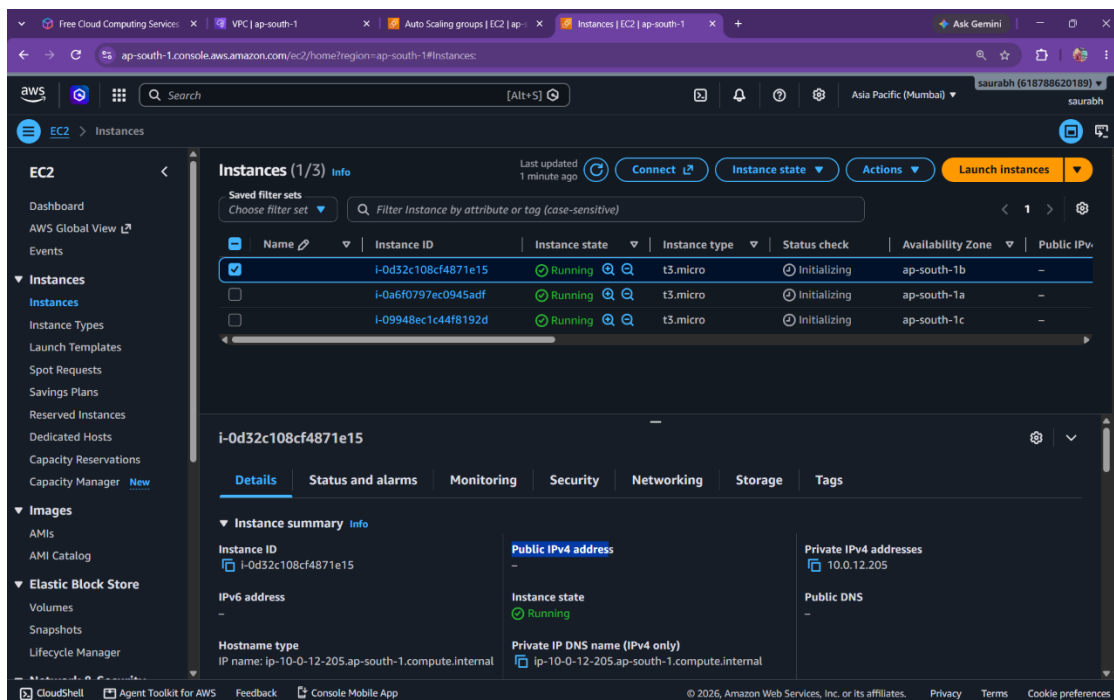
Step 30: EC2 Instances Launching

Three t3.micro EC2 instances came up automatically, one in each Availability Zone (ap-south-1a, 1b, 1c), confirming the Auto Scaling Group's multi-AZ distribution worked as configured.



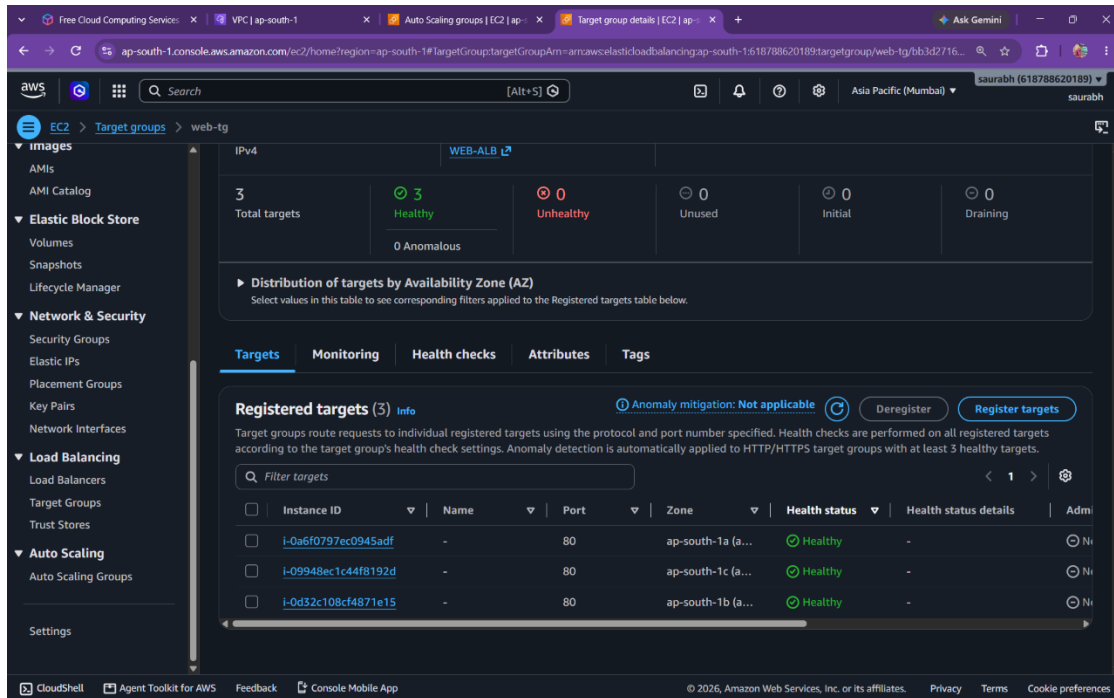
Step 31: Verify Instance Placement (Private IP)

Inspecting an instance confirms it received only a private IPv4 address (e.g. 10.0.12.205) and no public IP — proving it is correctly isolated inside a private subnet.



Step 32: Confirm Healthy Targets

In the web-tg target group, all 3 registered EC2 instances report a **Healthy** status, meaning the Load Balancer's health checks are passing and traffic can be routed to them.

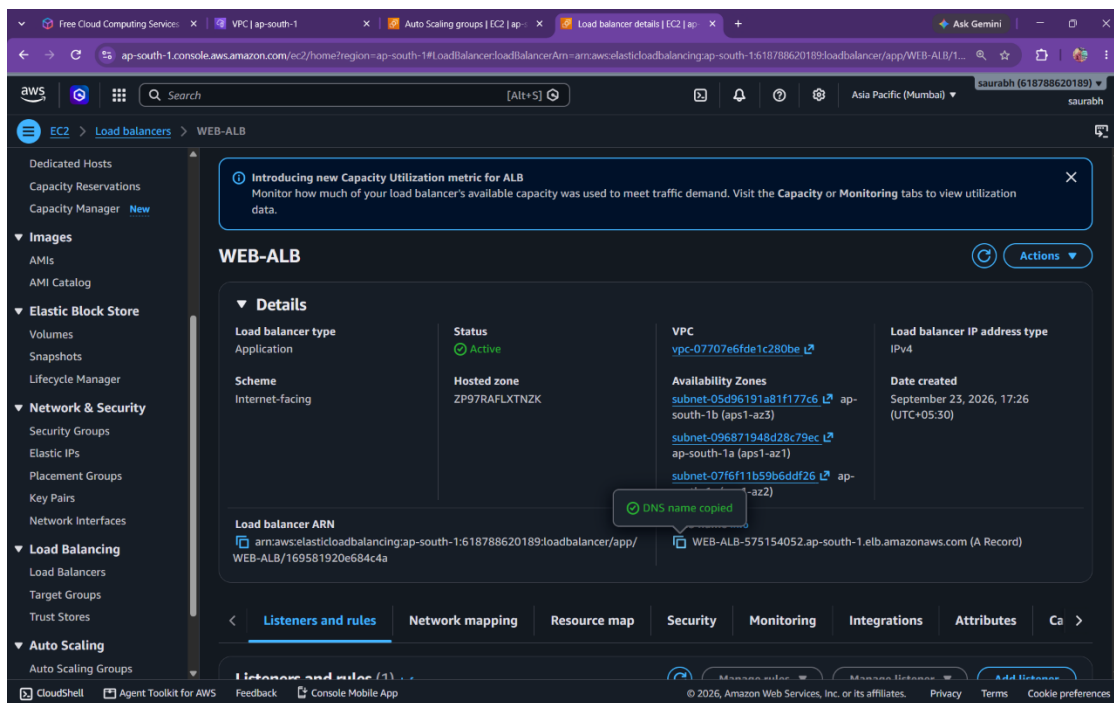


The screenshot displays the AWS Management Console interface for the 'Target group details' of a 'web-tg' target group. The console shows a summary of 3 total targets, all of which are 'Healthy'. Below this, a table lists the registered targets with their Instance IDs, Names, Ports, Zones, and Health Status.

Instance ID	Name	Port	Zone	Health status	Health status details
i-0a6f0797ec0945adf	-	80	ap-south-1a (a...)	Healthy	-
i-09948ec1c44f8192d	-	80	ap-south-1c (a...)	Healthy	-
i-0d32c108cf4871e15	-	80	ap-south-1b (a...)	Healthy	-

Step 33: Load Balancer Active

The WEB-ALB load balancer shows status **Active** with its public DNS name (WEB-ALB-575154052.ap-south-1.elb.amazonaws.com) ready to accept traffic across the three public subnets.



The screenshot displays the AWS Management Console interface for the 'Load balancer details' of a 'WEB-ALB' load balancer. The console shows the load balancer is 'Active' and provides details such as VPC, Availability Zones, and the public DNS name.

WEB-ALB Details:

- Status:** Active
- Hosted zone:** ZP97RAFLXTNZK
- VPC:** vpc-07707e6fde1c280be
- Availability Zones:** subnet-05d96191a81f177c6 (ap-south-1b (aps1-az3)), subnet-096871948d28c79ec (ap-south-1a (aps1-az1)), subnet-07f6f11b59b6ddf26 (ap-south-1c (aps1-az2))
- Date created:** September 23, 2026, 17:26 (UTC+05:30)
- Load balancer ARN:** arn:aws:elasticloadbalancing:ap-south-1:618788620189:loadbalancer/app/WEB-ALB/169581920e684c4a
- Public DNS name:** WEB-ALB-575154052.ap-south-1.elb.amazonaws.com (A Record)

Step 34: Test Load Balancing (Request 1)

Opening the ALB's DNS name in a browser returns a simple page identifying the backend server, e.g. "Server: ip-10-0-13-201", proving the ALB is successfully forwarding traffic to a backend EC2 instance.



Step 35: Test Load Balancing (Request 2)

A repeated request is served by a different instance, "Server: ip-10-0-11-221", demonstrating that the ALB is load-balancing traffic across multiple healthy targets rather than always hitting the same server.



Step 36: Test Load Balancing (Request 3)

A further request cycles back to "Server: ip-10-0-13-201", confirming consistent round-robin distribution of traffic across the Auto Scaling Group instances via the Load Balancer.



Server: ip-10-0-13-201

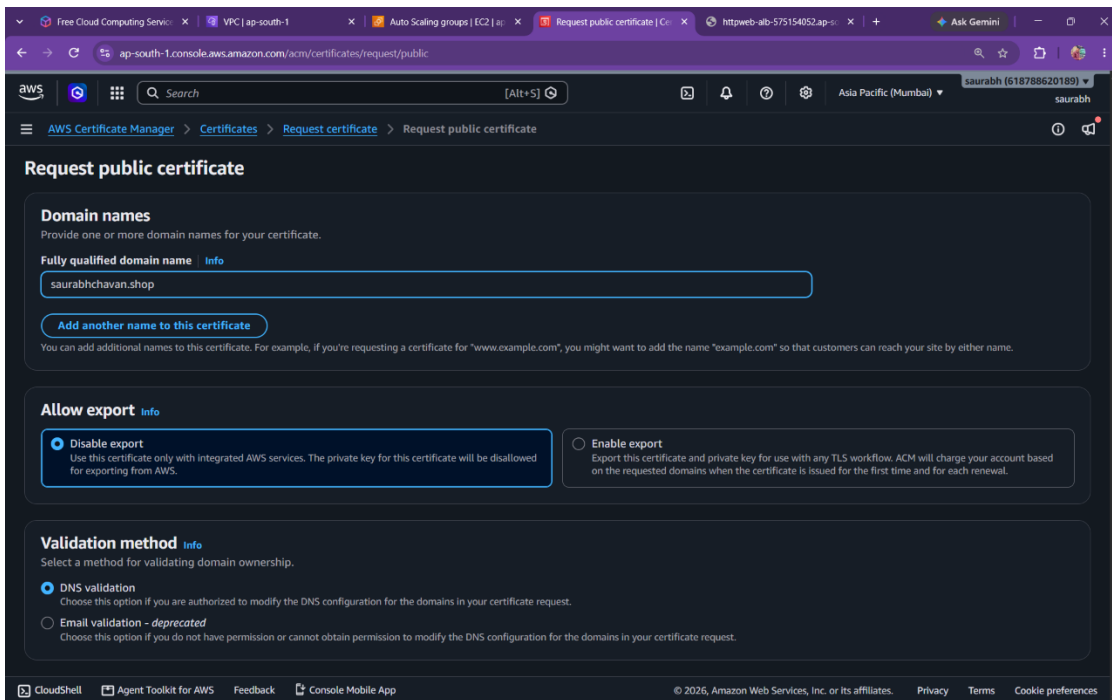


Phase 3: Custom Domain & HTTPS (Route 53 + ACM)

The final phase makes the application accessible on a friendly, secure custom domain by requesting a TLS certificate, adding an HTTPS listener to the Load Balancer, and pointing DNS at the ALB.

Step 37: Request a Public SSL/TLS Certificate

Using AWS Certificate Manager (ACM), a public certificate was requested for the domain **saurabhchavan.shop**, using DNS validation to prove domain ownership.



The screenshot shows the AWS Certificate Manager console in the 'Request public certificate' step. The browser tabs include 'Free Cloud Computing Service', 'VPC | ap-south-1', 'Auto Scaling groups | EC2 | ap-south-1', 'Request public certificate | Certificates | ap-south-1', and 'httpweb-alb-575154052.ap-south-1'. The console breadcrumb is 'AWS Certificate Manager > Certificates > Request certificate > Request public certificate'.

Request public certificate

Domain names
Provide one or more domain names for your certificate.

Fully qualified domain name [Info](#)

saurabhchavan.shop

[Add another name to this certificate](#)

You can add additional names to this certificate. For example, if you're requesting a certificate for "www.example.com", you might want to add the name "example.com" so that customers can reach your site by either name.

Allow export [Info](#)

☒ **Disable export**
Use this certificate only with integrated AWS services. The private key for this certificate will be disallowed for exporting from AWS.

☐ **Enable export**
Export this certificate and private key for use with any TLS workflow. ACM will charge your account based on the requested domains when the certificate is issued for the first time and for each renewal.

Validation method [Info](#)

Select a method for validating domain ownership.

☒ **DNS validation**
Choose this option if you are authorized to modify the DNS configuration for the domains in your certificate request.

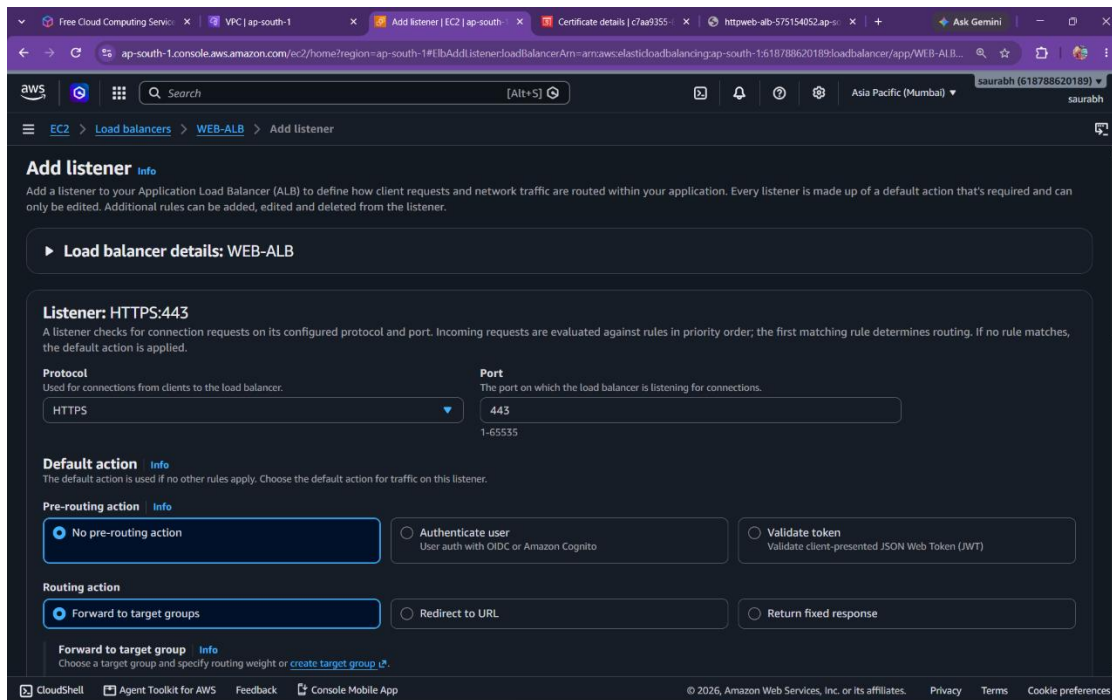
☐ **Email validation - deprecated**
Choose this option if you do not have permission or cannot obtain permission to modify the DNS configuration for the domains in your certificate request.

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

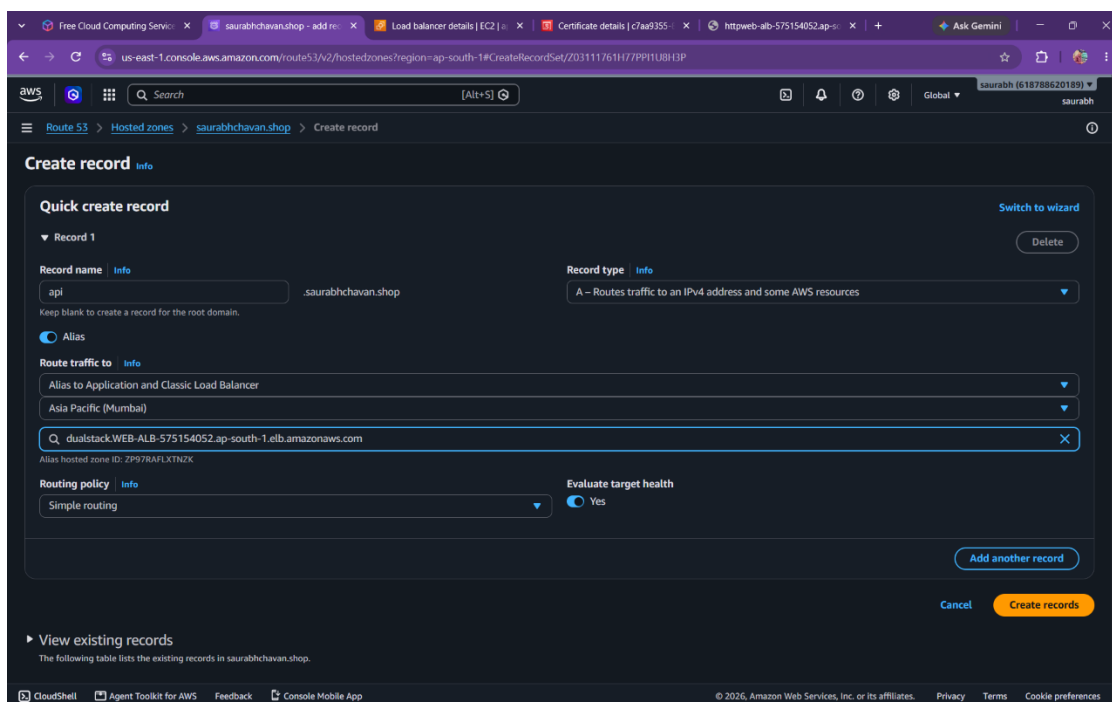
Step 38: Add an HTTPS Listener to the Load Balancer

An HTTPS listener on port **443** was added to WEB-ALB, using the validated ACM certificate and forwarding traffic to the web-tg target group — enabling encrypted traffic in addition to HTTP.



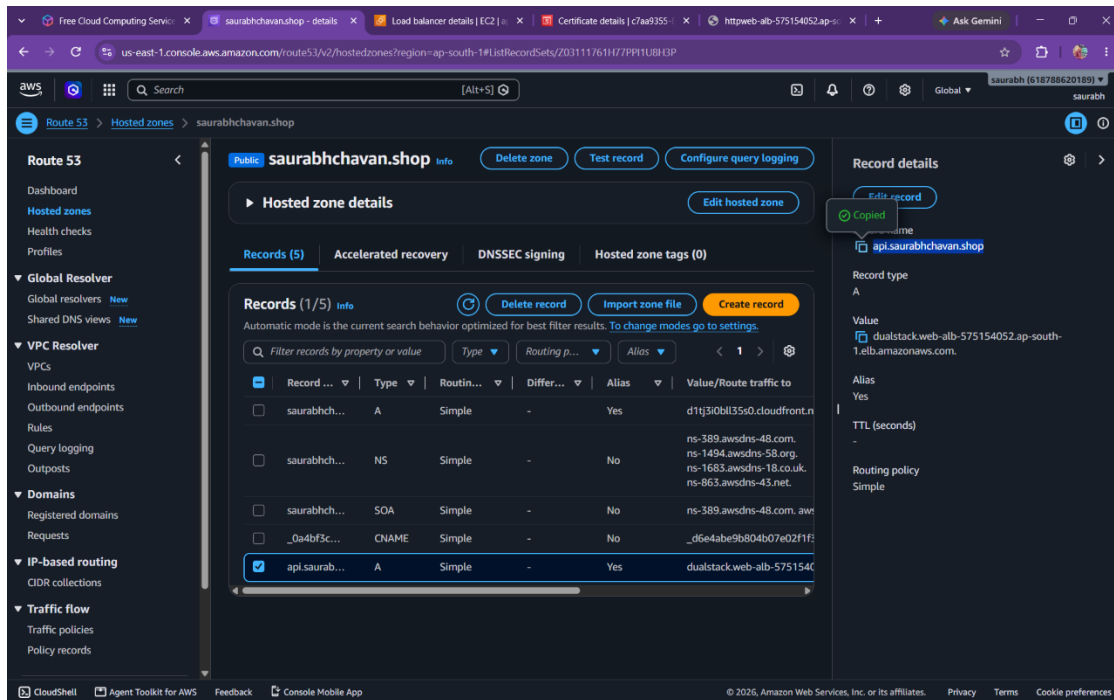
Step 39: Create a Route 53 Alias Record

In the saurabhchavan.shop hosted zone, an **A record (Alias)** named **api** was created, pointing to the WEB-ALB load balancer, so requests to **api.saurabhchavan.shop** resolve directly to the ALB.



Step 40: Confirm the DNS Record

The hosted zone record list confirms the new **api.saurabhchavan.shop** alias A-record pointing to "dualstack.web-alb-575154052.ap-south-1...", completing the DNS setup.



The screenshot shows the AWS Route 53 console for the hosted zone **saurabhchavan.shop**. The **Records** tab is active, displaying a list of 5 records. The record for **api.saurabhchavan.shop** is highlighted, showing it is an A record pointing to **dualstack.web-alb-575154052.ap-south-1.elb.amazonaws.com**.

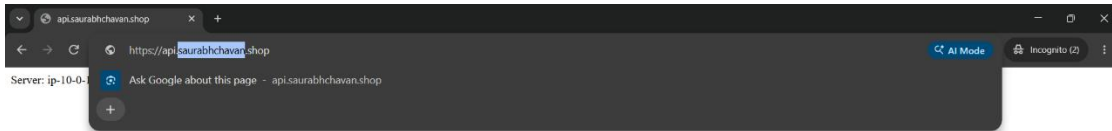
Record	Type	Routing	Difference	Alias	Value/Route traffic to
saurabhch...	A	Simple	-	Yes	d1tj3i0bl35s0.cloudfront.n...
saurabhch...	NS	Simple	-	No	ns-389.awsdns-48.com. ns-1494.awsdns-58.org. ns-1683.awsdns-18.co.uk. ns-863.awsdns-43.net.
saurabhch...	SOA	Simple	-	No	ns-389.awsdns-48.com. aw...
_0a4bf3c...	CNAME	Simple	-	No	_d6e4abe9b804b07e02f1f...
api.saurab...	A	Simple	-	Yes	dualstack.web-alb-5751540...

Phase 4: End-to-End Verification

With DNS and HTTPS configured, the application was tested using the final custom domain to confirm the entire request path — DNS → ALB → Target Group → EC2 — works correctly and continues to load-balance across instances.

Step 41: Browse to the Custom Domain

Navigating to **<https://api.saurabhchavan.shop>** successfully resolves via Route 53 and loads the application through the secure HTTPS listener.



Step 42: Verify Response from Backend Instance A

The page loads and identifies the responding backend, "Server: ip-10-0-12-205", confirming the domain is correctly wired to the Load Balancer and an application instance.



Step 43: Verify Load Balancing on the Custom Domain

A subsequent request to the same domain is served by a different backend, "Server: ip-10-0-11-221", confirming that load balancing continues to work correctly through the custom domain and HTTPS listener.



This concludes the deployment: a secure, load-balanced, auto-scaling, multi-AZ web application reachable at <https://api.saurabhchavan.shop>

Summary

This project successfully demonstrates a complete, production-grade AWS deployment pattern: a custom multi-AZ VPC with public/private subnet separation, redundant NAT Gateways, an Auto Scaling Group of EC2 instances behind an Application Load Balancer, and a custom domain secured with HTTPS via Route 53 and AWS Certificate Manager. The architecture is fault-tolerant (no single point of failure across AZs), elastic (auto scales between 1 and 7 instances based on demand), and secure (private compute tier, scoped security groups, and end-to-end HTTPS).